MSBDA-801 Big Data Analytics -- Final Project Report

# Scalable Real-time Detection Of AI-Generated Arabic Text

## A Distributed Big Data Pipeline Approach

Laila Abdullah Al-Mohaymid

ID: 4725094

# Abstract

The emergence of LLM in education and workplace has created a new challenge in discriminating human-written Arabic texts from AI-generated ones. The availability of LLM in academic and professional settings has created a new challenge that concerns the distinction between human-generated Arabic text and LLM-generated Arabic text. This project aims to remedy that by building a distributed end-to-end Big Data pipeline for processing Arabic academic abstracts at scale and classify them in real time. Built with Apache Spark 3.5.1, the distributed computation system, HDFS, the persistent columnar storage system in Parquet format, and Apache Kafka, the live stream ingestion system – this is the Lambda Architecture pattern using a unified system of batch and speed processing.

The dataset is KFUPM-JRCAI/arabic-generated-abstracts (Al-Shaibani and Ahmed, 2025), which includes 41,940 samples generated by 4 Arabic LLMs: ALLaM, JAIS, LLaMA 3.1 and GPT-4 along with original human-written abstracts. The four stylometric features that were engineered were word length-frequency distribution (f15), average sentences per paragraph (f36), top-50 embedding word count (f57), and an entropy-based perplexity score (f78). These were merged with TF-IDF representation obtained via Spark's MLlib. The three classifiers were compared, with Logistic Regression achieving the best performance (96.7% accuracy and 99.0% AUC-ROC on the held-out test set) and a scalability test demonstrating a speedup of 4.24x with increasing four processing cores.

***Keywords*:** AI-generated Arabic text detection; Big data analytics; Apache Spark; Lambda architecture; Real-time processing; Stylometric features; Arabic large language models.

# TABLE OF CONTENTS

# 1 INTRODUCTION

At first glance, the task of determining if a piece of text was written by a human or generated by a language model is simple, but in reality it is surprisingly difficult, especially for Arabic. Arabic is a morphologically rich right-to-left language, in which a single root may have dozens of surface forms and many stop words have considerable grammatical weight, not found in English. The transfer of English-based detection procedures is, therefore, mostly ineffective due to these characteristics.

The problem is much more pressing than ever before with the advent of Arabic capable LLM's like ALLaM, JAIS and GPT-4. Academic institutions have already seen plagiarized papers being submitted, which are generated by AI and sophisticated enough to evade traditional plagiarism detection tools, both in Saudi Arabia and globally throughout the Arab world. However, very little research has been conducted on Arabic-specific detection tools and almost none at the Big Data scale and in real-time.

It was this gap that led to this project. The challenge was not to build a single classifier but to create a production-oriented pipeline: to take a stream of Arabic texts from Kafka, preprocess them with distributed Spark UDFs, derive both stylometric and TF-IDF features and return a classification result in mere seconds. The architecture is suitable for periodic model retraining as new data comes in, as the same pipeline trains and evaluates three Spark MLlib classifiers on the entire 41,940-sample data set.

Specifically, the project objectives were: (1) install a distributed environment for HDFS, Spark and Kafka on CentOS Linux; (2) create an Arabic-specific pre-processing pipeline with Spark UDFs and ISRI stemming; (3) engineer four stylometric features based on the course position (i=15, n=21) and TF-IDF using Spark MLlib; (4) train and tune Logistic Regression, Random Forest and Linear SVM; (5) implement the best model in a Kafka + Spark Structured Streaming pipeline with a measurable scalability benchmark.

# 2 RELATED WORK

## 2.1 DETECTING AI-GENERATED TEXT

DetectGPT (Mitchell et al., 2023) is the earliest practical detector for machine-generated text at scale, that makes use of the fact that LLM-generated text is situated at local maxima of the log-probability floor of the model. On GPT-NeoX-generated news articles, DetectGPT reached 0.95 AUROC without any labeled training data -- an impressive zero-shot result. Yet, the method demands white-box access to the model's log probabilities, which is not possible with closed models, such as GPT-4 or ALLaM.

An alternative route are stylometric approaches. Kumarage et al. (2023) demonstrated that one can achieve reasonable accuracy in detecting AI-generated tweets by merely analyzing some surface-level writing style attributes, such as the frequency of sentence lengths, punctuation marks, and lexical diversity, without any access to models. This type of work is especially relevant for Arabic because there are sound definitions for the lexical diversity measures like TTR and Yule's K in terms of computation.

Al-Shaibani and Ahmed (2025) introduce the KFUPM-JRCAI dataset and perform the first large-scale stylometric analysis of Arabic LLM outputs. The feature selection for this project is directly influenced by their key finding that AI-generated Arabic text has lower vocabulary diversity and a greater Zipfian drop-off in the less common vocabulary. The pipeline does not support for distributed processing and real-time inference, they report up to 99.9% F1-score in controlled settings.

## 2.2 OVERVIEW OF BIG DATA ARCHITECTURES FOR NLP

Apache Spark is the go-to solution for scale processing of text. Spark's distributed machine learning library (MLlib) is efficient implementations of TF-IDF, logistic regression, random forest, and linear SVMs, which scale linearly across executors, described by Meng et al. (2016). Kocaman and Talby (2021) take this one step further by producing Spark NLP - which is production grade Arabic and multilingual NLP pipelines across hundreds of languages, in cluster environments.

This project is structured using the Lambda Architecture (Kiran et al., 2015). Kiran et al. show in their IEEE Big Data paper how the pattern can be implemented using Hadoop for the batch layer and a streaming framework for the speed layer, and how the separation of concerns can be used to handle both historical and real-time workloads cost effectively on Amazon EC2. In the messaging backbone, Kreps et al. (2011) proposed Apache Kafka, a fault-tolerant high throughput distributed log that has become the de facto standard for ingestion of data streams in data pipelines of this type.

# 3 DATASET DESCRIPTION

The dataset employed in the study throughout this project is KFUPM-JRCAI/arabic-generated-abstracts introduced by Al-Shaibani and Ahmed (2025) and publicly available on HuggingFace. It was built by removing Arabic academic papers from institutional repositories, abstracts, and titles were extracted and synthetic versions of them generated using four state-of-the-art LLMs, with different prompting strategies.

## 3.1 GENERATION STRATEGIES

| Method | Papers | Description |
|---|---|---|
| by_polishing | 2,851 | The LLM is given the original human abstract and asked to rewrite or refine it |
| from_title | 2,963 | The LLM generates a full abstract given only the paper title |
| from_title_and_content | 2,574 | The LLM uses both title and body content as context for generation |
| Total | 8,388 | Unique papers, each paired with four AI-generated versions |

## 3.2 LABEL DISTRIBUTION AFTER FLATTENING

The subsequent step is to distribute the labels. What follows is the distribution of the labels after flattening.

The last corpus is 41,940 samples for the dataset after converting to long format (one sample per text). This class imbalance (1:4 human:AI) is simply the structure of the dataset: every human abstract is matched with four AI counterparts, one for each model. This imbalance was considered when evaluating the models.

| Label | Count | Percentage | Source Models |
|---|---|---|---|
| Human | 8,388 | 20.0% | Original human-authored abstracts |
| AI | 33,552 | 80.0% | ALLaM, JAIS, LLaMA 3.1, GPT-4 |
| Total | 41,940 | 100% | Split 70/15/15: Train 29,436 \| Val 6,197 \| Test 6,303 |

# 4 METHODOLOGY

## 4.1 DISTRIBUTED ENVIRONMENT (PHASE 1)

All of the pipeline is deployed on a single CentOS 10 Linux node (aarch64 architecture), with Apache Spark 3.5.1, Hadoop 3.x as an HDFS service, and Apache Kafka 3.7.0 as a stream processing service. A single-node setup is not a multi-rack cluster but it can be used to demonstrate the Lambda Architecture end to end and can have a valid baseline for scalability benchmarking.

The raw dataset was downloaded from HuggingFace via the HuggingFace datasets library, then quickly converted from wide to long format in Python, and then directly written to HDFS as a snappy compressed Parquet file (22MB disk size). The processing workflow is completely independent of the data acquisition step as subsequent pipeline stages only read from HDFS.

## 4.2 ARABIC PREPROCESSING USING SPARK UDFS (PHASE 2)

All pre-processing was done as UDFs in Spark, to be processed in parallel over different Spark partitions instead of being processed by the driver. The pipeline consists of four stages: first, it removes diacritics from every document (Unicode range U+0617-U+0652 by regex), then it normalizes the Arabic text by unifying all Alefs (also Ta Marbuta and Alef Maqsura), then it filters out stop words from the text using an 804-word Arabic lexicon, and lastly it stems the text by using the ISRI Arabic Stemmer (Taghva et al., 2005). The resulting corpus was written back to HDFS as another Parquet file.

Before diving into the higher-level MLlib abstractions, two MapReduce style jobs were implemented with Spark RDDs to compute statistics on the corpus. Job 1 associated each token with (word, 1) and reduced by summing, resulting in the entire word frequency distribution, which had the highest frequency stem of 'daras' (study) with 72,760 occurrences. Job 2 extracted pairs of consecutive words, mapped the same way and output the bigram frequency table, with 'hadaf daras' (study objective) being the most common 11,624 times.

## 4.3 FEATURE ENGINEERING (PHASE 3)

The formula for feature assignment was: f_(kn+i) where i=15 (student position) and n=21 (class size), which gave four features at position 15, 36, 57, and 78.

| ID | Feature | Human | AI | Notes |
|---|---|---|---|---|
| f15 | Word Length-Frequency Distribution | 1.439 | 1.184 | Computed as variance over word length counts; human text more varied |
| f36 | Avg Sentences per Paragraph | 9.504 | 8.576 | Paragraphs split on newlines; human abstracts structurally denser |
| f57 | Top-50 Embedding Words Count | 6.841 | 10.151 | Proxy using corpus top-50 stems; AI uses high-frequency vocab more |
| f78 | Perplexity Score (entropy-based) | 5.789 | 5.497 | Shannon entropy over unigram distribution; human text less predictable |

There are two features that require a note of implementation. The specification for f15 requires a full-length word length-frequency distribution (variable-length histogram). VectorAssembler takes numeric vector inputs, which are required to be of fixed length, so we made a scalar summary of the distribution of word lengths, a standard scalar reduction of the distribution, called variance. We chose the top-50 most frequent corpus stems as a frequency based approximation in the case of f57, due to the memory limitations of the single node environment. Both approximations are mentioned as limitations in Section 7.

TF-IDF: The advanced feature is computed using HashingTF + IDF pipeline from Spark MLlib with 5,000 hash buckets and unigram+bigram tokenization. Features were combined by the VectorAssembler to obtain a single dense feature vector per document from the following features: stylometric and TF-IDF.

## 4.4 MODEL TRAINING (PHASE 3)

The three classifiers were trained with the 70% split: Logistic Regression (maxIter=100, regParam=0.01) was required by the Task 3.4; random forest (numTrees=100) and linear SVM (maxIter=100) were the advanced models for the Task 3.5. The results of all models were tested on the held-out 15% test partition (n=6,303 samples) with the help of BinaryClassificationEvaluator and MulticlassClassificationEvaluator modules. The model that performed best (Logistic Regression) was saved in the local directory models/ using the MLlib's native save API.

## 4.5 REAL-TIME PIPELINE (PHASE 4)

There are two parts to the streaming pipeline. A Python Kafka producer reads 100 samples from the raw Parquet file, serializes each as a JSON payload containing the text, true label and a record ID, and publishes them to the 'arabic-texts' Kafka topic every 0.5 seconds -- this is a live submission feed. This Spark Structured Streaming consumer also subscribes to this topic, parses the JSON schema, applies the same UDFs that were applied during batch training, categorizes each micro-batch with the lightweight scoring logic, and writes the results to the console with a time stamp. After 100 messages had been sent through the pipeline, all of them were received without missing, and the pipeline ran for 90 seconds.

# 5 RESULTS AND ANALYSIS

## 5.1 MODEL PERFORMANCE ON THE TEST SET

| Model | Accuracy | F1-Score | Precision | Recall | AUC-ROC |
|---|---|---|---|---|---|
| **Logistic Regression (*)** | **96.70%** | **96.71%** | **96.72%** | **96.70%** | **99.05%** |
| Linear SVM | 95.97% | 96.04% | 96.23% | 95.97% | 98.33% |
| Random Forest | 79.80% | 70.84% | 63.69% | 79.80% | 94.72% |

Logistic Regression and Linear SVM showed comparable performance with LR having a slight but consistent advantage on all the metrics. The AUC-ROC score for the combined TF-IDF + stylometric feature space is near perfect (0.99), which means that the space is very separable for this task and LR's linear decision boundary is enough.

Random Forest's failure is more instructive. The confusion matrix it produced looked like a "degenerate pattern": all 6,303 test samples were correctly identified as 'human' (5,030 were correct, 1,273 were AI texts misclassified). This is typical of ensemble tree methods when dealing with a high dimensional sparse feature space (the 5000 dimensional tf-idf feature vector dominates the tree splitting process, which tends to focus on the majority class). In future work, this issue would probably be solved by class weight or upsampling the minority class.

## 5.2 FEATURE IMPORTANCE ANALYSIS

Random Forest feature importances, although not very predictive of the model, are indicative of which features contain discriminative information. The top-50 Embedding Words (f57, importance 0.0531) feature was the most prominent feature, higher than the individual TF-IDF dimensions. This is closely in line with Al-Shaibani and Ahmed's (2025) result that the high-frequency vocabulary was overused in the AI-generated Arabic texts, with the average number of top-50 words being 10.15 compared to 6.84 in human-written texts, a relative increase of 48%.

| Rank | Feature | Importance | Category |
|---|---|---|---|
| **1** | **f57 -- Top-50 Embedding Words** | **0.0531** | **Stylometric** |
| 2 | TF-IDF (index 4211) | 0.0391 | TF-IDF |
| 3 | TF-IDF (index 1085) | 0.0331 | TF-IDF |
| 4 | TF-IDF (index 69) | 0.0317 | TF-IDF |
| ... | f15 -- Word Length Variance | 0.0180 | Stylometric |
| ... | f78 -- Perplexity Score | 0.0142 | Stylometric |
| ... | f36 -- Avg Sentences/Paragraph | 0.0098 | Stylometric |

## 5.3 VOCABULARY RICHNESS (EDA)

| Metric | Human | AI | Observation |
|---|---|---|---|
| Type-Token Ratio (TTR) | 0.7585 | 0.7248 | Human text 4.7% richer in vocabulary |
| Avg Text Length (chars) | 378.4 | 333.2 | Human abstracts roughly 14% longer |
| Word Length Variance (f15) | 1.439 | 1.184 | Human word-length distribution wider |
| Perplexity Score (f78) | 5.789 | 5.497 | Human text marginally less predictable |

## 5.4 SCALABILITY BENCHMARK (TASK 4.4)

To compare the scalability of processing time with the number of cores, the same data (41,936 rows) was processed three times with different number of cores;

| Cores | Time (s) | Speedup | Notes |
|---|---|---|---|
| 1 | 2.50 | 1.00x | Baseline -- sequential execution |
| 2 | 0.67 | 3.73x | Near-linear speedup with 2 threads |
| 4 | 0.59 | 4.24x | Diminishing returns at 4 cores (I/O bound) |

The results indicate that the performance scales almost linearly from 1 to 2 cores (3.73x), and decreases at 4 cores (4.24x). This is normal: It is not the CPU that is the limiting factor anymore, it is the HDFS read I/O. If the HDFS blocks were distributed across the nodes in a true multi-node cluster, the scaling curve would be straighter.

## 5.5 BATCH VS. STREAM PROCESSING TRADE-OFFS

A big disadvantage of batch processing is that latency -- a new text cannot be classified until the next training cycle.Batch processing (Spark MLlib on the full 41,940 samples) offers high throughput and can be used to train complex models, but has the drawback of latency -- a new text cannot be classified until the next training cycle. A classification logic was implemented with stream processing (Kafka + Spark Structured Streaming) with almost real time classification, with a simplified classification logic (less of the Logistic Regression pipeline, because loading an MLlib model inside a streaming UDF requires additional serialization).

This tension is addressed in a production deployment: Periodically, the batch layer will retrain and export the model, and the speed layer will apply the latest exported model to the incoming messages. This design has been shown conceptually in this project: The batch model was saved to models/, and the streaming pipeline used the same feature logic.

# 6 CONCLUSION AND FUTURE WORK

The aim of this project was to develop a scalable real-time Arabic text detection pipeline for AI-generated Arabic text and they managed to create an end-to-end system that achieved 96.7% accuracy and 99.0% AUC-ROC on a benchmark set of 41,940 samples. There are some noteworthy findings.

The result is surprisingly good that the combination of TF-IDF and simple stylometric features is quite effective.First, this combination of TF-IDF and simple stylometric features is very effective. In this academic field, Logistic Regression with a linear boundary can separate the human and AI classes effectively, with an accuracy of almost 100%, indicating that the two classes are in large part mutually exclusive in terms of the distribution of features. Second, the f57 feature is the most informative stylometric predictor, as found in the work of Al-Shaibani and Ahmed (2025) which

reports the vocabulary diversity gap. Third, Random Forest's performance is disastrous when the classes are imbalanced and TF-IDF features are sparse, which is a potential warning for future implementation of class-weighted training and/or oversampling.

Infrastructure-wise, the Spark pipeline achieved a near linear performance increase from 1 to 2 cores (3.73x speedup) and I/O scaling limited the performance at 4 cores. The Kafka streaming component took on 100 real-time messages at sub-second latency per micro-batch.

## Limitations

There are three restrictions that need to be made explicit. First, it is not a true multi-rack cluster deployment, but the scalability results are indicative, not definitive. Second, f15 is a scalar approximation of the full word-length distribution, and f57 is a corpus-frequency approximation of word embedding rank -- both approximations used due to limited memory. Third, the streaming classifier has simplified feature logic; it would be necessary to implement the full MLlib model into the streaming pipeline, which involves more engineering.

## Future Work

The most useful next steps would be: Deploying on a multi-node Spark cluster (such as three workers) to validate the claims of scalability; Integrating the entire Logistic Regression model into the streaming pipeline, using the PipelineModel.load() function from MLlib; Applying class weighted training or SMOTE to regain some performance of the Random Forest model; Testing cross domain generalization on the KFUPM-JRCAI social media dataset. In the longer term, fine-tuning an Arabic BERT model, either AraBERT or CAMeL-BERT, in a Spark NLP pipeline would likely further boost the accuracy to exceed 99%.

# 7 REFERENCES


[1] Al-Shaibani, M. S., and Ahmed, M. (2025). The Arabic AI Fingerprint: Stylometric Analysis and Detection of Large Language Models Text. arXiv preprint arXiv:2505.23276. SDAIA-KFUPM Joint Research Center for Artificial Intelligence, King Fahd University of Petroleum and Minerals. Available: https://arxiv.org/abs/2505.23276

[2] Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., and Finn, C. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), Honolulu, Hawaii. arXiv:2301.11305. Available: https://arxiv.org/abs/2301.11305

[3] Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D. B., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M. J., Zadeh, R., Zaharia, M., and Talwalkar, A. (2016). MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research, 17(34), 1-7. Available: https://arxiv.org/abs/1505.06807

[4] Kiran, M., Murphy, P., Monga, I., Dugan, J., and Baveja, S. S. (2015). Lambda Architecture for Cost-Effective Batch and Speed Big Data Processing. In Proceedings of the 2015 IEEE International Conference on Big Data, Santa Clara, CA, USA, pp. 2785-2792. IEEE. Available: https://ieeexplore.ieee.org/document/7364082

[5] Kocaman, V., and Talby, D. (2021). Spark NLP: Natural Language Understanding at Scale. Software Impacts, 8, 100058. arXiv:2101.10848. Available: https://arxiv.org/abs/2101.10848

[6] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012), San Jose, CA. Available: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

[7] Kreps, J., Narkhede, N., and Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. In Proceedings of the Workshop on Networking Meets Databases (NetDB) at SIGMOD 2011, Athens, Greece. ACM.

[8] Kumarage, T., Garland, J., Bhattacharjee, A., Trapeznikov, K., Ruston, S., and Liu, H. (2023). Stylometric Detection of AI-Generated Text in Twitter Timelines. arXiv preprint arXiv:2303.03697. Available: https://arxiv.org/abs/2303.03697

[9] Taghva, K., Elkhoury, R., and Coombs, J. (2005). Arabic Stemming without a Root Dictionary. In Proceedings of the International Symposium on Information Technology: Coding and Computing (ITCC 2005), Las Vegas, NV, vol. 1, pp. 152-157. IEEE. Available: https://ieeexplore.ieee.org/document/1427520